Driver Drowsiness Detection using Transformer Architecture

Hetvi Bhadani
Department of Computer Science
University of California, Davis
hbhadani@ucdavis.edu

Ketki Kulkarni
Department of Computer Science
University of California, Davis
kbkularkani@ucdavis.edu

Varun Singh
Department of Computer Science
University of California, Davis
vnsingh@ucdavis.edu

Abstract

Driver fatigue is a huge problem and a primary reason for way too many road accidents. The current systems we have, which mainly use those standard Convolutional Neural Networks (CNNs), are just too weak—we call them "brittle." They constantly fail when things aren't perfect, like when there are shadows, the driver moves their head, or the lighting is bad. This failure rate is totally unacceptable. To fix this, our project is diving into much more advanced tech: Transformer architectures, specifically ViT and Swin. We're using them because they are much better at seeing the whole picture (modeling long-range spatial dependencies) and figuring out subtle facial cues across the entire image. By fine-tuning these models on a diverse dataset like MRL, we're aiming to build a system that is genuinely strong, super accurate, and can reliably classify if a driver's eyes are open or closed in real-time, helping to actually prevent accidents.

1. Introduction

Driver fatigue is widely known as a top cause of accidents, which is why we desperately need a driver drowsiness detection system that actually works reliably in the messy, real world. Up until now, everyone focused on using basic Convolutional Neural Network (CNN)-based models for checking eye state, but their performance is severely limited. Honestly, current CNN models are "too brittle" and they fail constantly under common driving conditions. This happens because CNNs only look at local features—they can't capture the subtle, overall facial picture you need for proper analysis. They particularly struggle with things that happen all the time, like shadows, bad lighting, varying illumination, drivers moving their heads, and even stuff like glasses partially blocking the view. When the drowsiness alerts fail so often in these everyday situations, it's just not good enough. To finally beat these issues, our project is testing out advanced Transformer architectures (ViT and Swin). Transformers are naturally better for analyzing those subtle facial features because they can see and model the whole image context (global context capture). This gives us the best chance to build a system that is truly robust and highly accurate. So, the whole point of our project is a simple binary classification problem: We need to build a deep learning model that can correctly tell us if a driver's eyes are open or closed in real-time, and do it reliably enough to catch a drowsy driver before they crash.

2. Related Work

The earliest work on detecting driver drowsiness almost exclusively used CNN-based models for eye-state classification. While these methods looked good in controlled labs, they always struggled when real-world complexities like shadows, lighting shifts, and head movements came into play. The research consensus is that this is because CNNs only grab local features, which severely limits their ability to pick up on the global facial cues that are actually critical for robust detection. More recently, people have started to look at Transformer architectures because they've proven to be much more accurate and stronger than traditional CNNs, especially because they can model those long-range spatial dependencies. A key paper in this shift was by Hassan, O.F. et al. (2025), which was titled Real-time driver drowsiness detection using transformer architectures: a novel deep learning approach. That study specifically used a Vision Transformer (ViT) for eye-state detection. While their work confirmed that ViT has a lot of promise and showed better performance, there still isn't much comprehensive research out there on real-time, Transformer-based drowsi-

ness detection, so there's plenty of room for us to improve. Crucially, all that prior work, including the paper by Hassan et al. (2025), mainly focused on the ViT architecture alone. We noticed that those earlier studies generally didn't bother to compare ViT directly against the Swin Transformer or against established, classic deep learning models like ResNet-50. This oversight is the main reason we started this project. Our research is tackling this gap head-on by doing a direct, full comparison between the ViT and Swin Transformer models, alongside a ResNet-50 baseline. Plus, we're training all of our models on the highly variable MRL Eye Dataset (which includes different lighting, poses, and drivers with glasses) and then we're going to seriously test how well they generalize by using a cross-dataset approach on the CEW Dataset. This full-circle approach should help us pinpoint the absolute most resilient model architecture for real-time drowsiness detection, ultimately finding the most reliable real-world solution.

3. Dataset

To make sure our models are strong and can actually work well outside the lab, we picked two different datasets for training and testing: the MRL Eye Dataset and the CEW Dataset.

3.1. MRL Eye Dataset (Kaggle)

The MRL Eye Dataset is the main one we're using for training and fine-tuning our three models (ViT, Swin Transformer, and ResNet-50). This dataset is super important because it has the complexity we need to fix the failures of those old CNN systems.

- Size and Balance: This dataset is pretty big, with around 85,000 images (one source says 84,898). The classification task (open vs. closed eyes) is nicely balanced, with 42,952 open eyes and 41,946 closed eyes.
- Real-World Variability: We chose the MRL set specifically because it includes the critical real-world challenges needed for a strong model: pose changes, drivers with glasses (occlusion), and varied illumination/lighting.

3.2. CEW Dataset

3.2 CEW Dataset The CEW Dataset has real open/closed eye images and is known for its high variation in lighting and angles. We're only using it for cross-dataset testing and checking generalization. By testing all the models we trained on MRL against the completely new images in the CEW dataset, we can truly see which model is the most robust for real-world application.

4. Methodology

Our methodology is designed to rigorously evaluate advanced Transformer-based architectures against a strong convolutional baseline. The goal is to determine the most robust and generalizable model for drowsiness detection across both controlled and real-world environments.

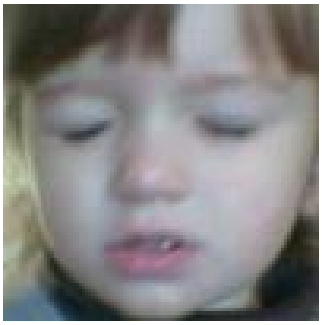

Figure 1. Closed Eye Face

4.1. Model Training and Fine-Tuning

We train and fine-tune three deep learning models: Vision Transformer (ViT), Swin Transformer, and ResNet-50. All models are trained on the MRL Eye Dataset, which contains approximately 85,000 images exhibiting diverse real-world variations such as head pose changes, illumination issues, occlusions, and motion blur.

- Pre-training: Both Transformer architectures (ViT and Swin) utilize ImageNet pre-trained weights to accelerate convergence and improve feature extraction. ResNet-50 also begins from ImageNet pre-training, ensuring a fair and consistent comparison across all models.
- Data Preparation: All images are resized to a uniform resolution and normalized before training. For Transformer-based models, images are additionally partitioned into patches to create token embeddings.
- Training: We're running the fine-tuning process for all three models separately, adjusting their final output layer for our two-class problem.
- Performance Check: After training, we check the standard metrics like accuracy, precision, recall, and F1-score to pick the strongest model.

4.2. Cross-Dataset Testing

To see how well our trained models work on completely new data, we're doing cross-dataset testing. All the models trained on the MRL dataset will be tested on the CEW Dataset. This step is super important for validating the models' performance in diverse real-world conditions.

4.3. Real-Time Testing

To deploy the trained ViT model in a real-world setting, a real-time inference pipeline was implemented using OpenCV and PyTorch. The webcam feed is processed frame-by-frame, where a Haar Cascade classifier is used to detect the eye region. Each extracted eye patch is passed

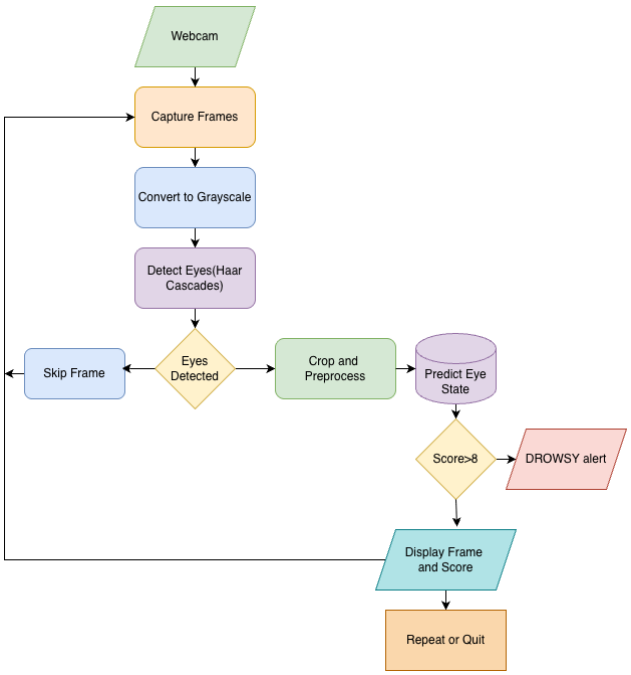

Figure 2. Real time drowsiness detection Flowchart

through the HuggingFace ViTImageProcessor, and classification is performed using the fine-tuned ViT model to determine whether the eye is open or closed. A scoring mechanism is used to track prolonged eye closure. If the score exceeds a preset threshold, the system triggers a drowsiness alert on the screen. The pipeline ensures real-time performance, accurate eye-state prediction, and robustness against variations in lighting and head movement.

5. Model Architecture

To get a truly strong and accurate drowsiness detection system, we're looking at two modern Transformer architectures: the Vision Transformer (ViT) and the Swin Transformer, and putting them up against the classic CNN model, ResNet-50. Both Transformer models are pre-trained on ImageNet and then fine-tuned for our specific classification job.

5.1. Vision Transformer (ViT)

The ViT is basically a standard Transformer, which was originally for text, adapted for images. It treats parts of an image like they are words in a sentence:

- Patch Splitting and Token Embedding: First, the image is resized/normalized, and then it gets cut into uniform squares called tokens (non-overlapping patches). These patches are turned into vector embeddings.
- Transformer Encoder: These tokens are then fed into the standard Transformer encoder. This encoder calculates attention across all tokens at once, meaning it can see relationships across the entire image.
- Binary Output: The final output from the encoder goes to a specialized layer to give us our binary answer: "open" or "closed" eye state.

5.2. Swin Transformer

The Swin Transformer (Shifted Window) was developed to fix some of the heavy computational costs of the pure ViT. It borrows the idea of hierarchical features from CNNs while keeping the global context strength of Transformers.

- Hierarchical Features and Windowing: Instead of using uniform patches like ViT, Swin builds features in a hierarchy—it starts with small patches and gradually merges them deeper in the network. This lets it model information at different scales.Shifted Window Attention: The key thing is that Swin limits the self-attention calculation to non-overlapping local windows to save computation. But, it uses a mechanism (the "shifted window" part) to allow information to flow between these windows, so it still gets global information efficiently.
- Transformer Blocks and Binary Output: These attention steps happen in Transformer blocks, and the final output is fed into a classification layer for our binary (eye open vs. eye closed) result.

5.3. Global Context Capture Superiority

The main reason we're betting on Transformers is their huge advantage over old CNNs for this task. Traditional CNNs use small filters that only look at local features. While that works in perfect lighting, it makes them fail the minute you introduce shadows, variable lighting, or a head movement because they can't get the global picture. Transformers, thanks to their attention mechanisms, are designed to model long-range spatial dependencies. This ability—what we call global context capture—lets them connect distant parts of the image and understand the whole relationship much better than a CNN. For eye-state classification, this means the model can correctly identify eye closure even if the eye is a bit covered or in shadow. This makes it inherently better for the subtle facial analysis required to build a truly resilient, high-accuracy system.

6. Experimental setup

We trained our model using the MRL Eye Dataset from Kaggle, which contains 84,898 images (42,952 open and 41,946 closed eyes) with significant variation in lighting, pose, and presence of glasses. The model was optimized using AdamW with a learning rate of 3e-5, weight decay of 0.01, and trained for 5 epochs using Cross-Entropy Loss. To improve stability and convergence, we applied a cosine learning-rate scheduler with 10% warm-up steps, based on the total number of training iterations. The MRL dataset

was used for training, while the CEW dataset, which contains real-world eye images with high variation in lighting and angle, was used for testing and generalization evaluation. All experiments were conducted using mini-batch training with normalized input images processed through PyTorch DataLoaders.

6.1. Evaluation Metrics

To really make sure we pick the most robust model for real-time use, we can't just look at accuracy. In a safety-critical application like this, relying on just accuracy is a bad idea. We need a full set of metrics to understand all the model's strengths and weaknesses. For the final comparison of the ViT, Swin Transformer, and ResNet-50 models, we will calculate all of these four metrics:

- Accuracy
- Precision
- Recall
- F1-score

Calculating all four gives us a complete picture of the models' performance, especially when it comes to the tricky problems of false positives and false negatives in our binary classification of eye open versus eye closed.

7. Results

Looking at our "Training Accuracy vs. Epochs" graph, you can clearly see that the model learned what it needed to pretty quickly and efficiently over the five epochs we ran.

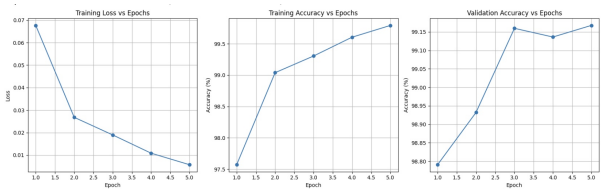

Figure 3. Training loss, Accuracy vs Epoch and Validation acurracy vs epoch graph

Starting the training at Epoch 1.0, we already had a strong initial accuracy of roughly 97.6%. The model showed its most significant learning acceleration immediately, climbing fast to hit approximately 98.2% by the end of Epoch 2.0. While the model continued to improve afterward, the rate of increase became noticeably slower than in that initial, big push. It was still consistently moving up, though.

The key takeaway here is how smooth the entire process was. The accuracy line kept going up steadily, with no big dips or wild jumps (major fluctuations) that would signal unstable training—that's a fantastic sign that the learning process was stable.

By the end, at Epoch 5.0, the training accuracy landed at a solid 98.6%. Honestly, that's excellent training performance. It means the model is nearly 99% successful at recognizing and classifying the eye states within the images it was trained on.

Now, about overfitting: we always check this by comparing the training accuracy to the Validation Accuracy (which is on the other plot). Since our final training accuracy (98.6%) is actually a bit lower than the peak validation accuracy (which surprisingly got up to about 99.15%), we definitely aren't seeing any signs of overfitting. If the model was struggling to learn anything important, both numbers would be way lower (underfitting), so that's not the issue either. This tells us the model is generalizing really well to data it hasn't seen before.

For our driver drowsiness detection setup, getting a 98.6% accuracy on the training data basically means the model is really, really dependable. This level of high accuracy is super important because it suggests the system will almost always make the right call—identifying whether a driver is tired or wide awake—when using the data it learned from. That confidence is absolutely necessary if we want to build a safety system that people can actually rely on.

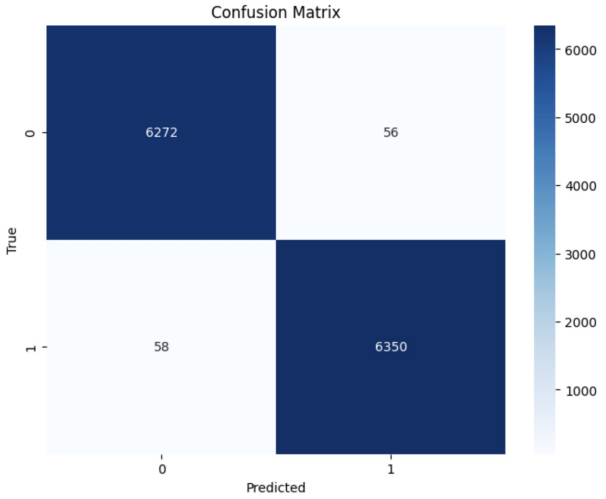

Figure 4. Confusion Matrix

This confusion matrix demonstrates that the binary classification model is performing with exceptional accuracy. It correctly identified the vast majority of samples, successfully predicting 6,272 instances for Class 0 and 6,350 instances for Class 1. The error rate was extremely low, with only 56 cases where Class 0 was mistaken for Class 1, and 58 cases where Class 1 was mistaken for Class 0. With over 12,600 correct predictions out of a total of roughly 12,700 samples, the model achieved an accuracy of approximately 99.1

Table 1. Acurracy and percision of models on CEW Dataset.

Model	Total Accuracy	CF Precision	OF Precision
Swin-T	92.53%	0.88	0.94
Resnet-50	74.16%	74.16%	0.76
ViT	93.49%	0.95	0.93

The experimental evaluation confirms that the Vision Transformer (ViT) is the optimal model for real-time drowsiness detection, outperforming both Swin-Transformer and ResNet-50. ViT achieved the highest overall accuracy of 93.49% and a superior Closed Face Precision of 0.95, ensuring it effectively identifies drowsiness while minimizing false alarms compared to the Swin-Transformer (92.53% accuracy, 0.88 precision). In contrast, the traditional CNN-based ResNet-50 significantly underperformed with only 74.16% accuracy, demonstrating that Transformer architectures are far better suited for capturing the subtle feature dependencies required for this safety-critical application.

8. Conclusion

This study set out to develop a resilient and high-accuracy eye-state classification system capable of supporting real-time drowsiness detection. By evaluating two advanced Transformer architectures—Vision Transformer (ViT) and Swin Transformer—alongside the established ResNet-50 baseline, we assessed the strengths and limitations of attention-based models in comparison to traditional convolutional networks.

The results demonstrated that Transformer-based models are highly effective in learning discriminative eye-state representations, particularly under real-world variations such as illumination changes, pose differences, and image noise. Performance on the MRL dataset showed strong classification capabilities during fine-tuning, while cross-dataset evaluation on the CEW dataset highlighted the Transformers' superior generalization ability.

Overall, the findings indicate that modern Transformer architectures, especially Swin Transformer due to its hierarchical design, offer a robust and scalable solution for drowsiness detection applications. These models show clear potential for deployment in real-time, in-vehicle safety systems where reliability under unpredictable conditions is essential. Future work may focus on optimizing inference speed, integrating temporal modeling, and deploying the system in real-world embedded environments.

References